Main.py

```
import socket
import threading
import os
import re
import pandas as pd
from PyPDF2 import PdfReader
from bs4 import BeautifulSoup

# Constants
SERVER_HOST = "127.0.0.1"
SERVER_PORT = 52300
BUFFER_SIZE = 1024
ENDING_MSG = "q"
END_MARKER = "<<END>>"
DIRECTORY = "./data"  # Directory containing files

def search_txt(file_path, word, logical_operator=None, regex=False):
    results = []
    file_name = os.path.basename(file_path)
    with open(file_path, "r", encoding="utf-8") as file:
        for line_no, line in enumerate(file, start=1):
            if regex:
                if re.search(word, line):
                    results.append(f"TXT - {file_name}, Line {line_no}:
{line.strip()}")
            elif logical_operator == "and":
                if all(w in line for w in word.split()):
                    results.append(f"TXT - {file_name}, Line {line_no}:
{line.strip()}")
            elif logical_operator == "or":
                if any(w in line for w in word.split()):
                    results.append(f"TXT - {file_name}, Line {line_no}:
{line.strip()}")
            else:
                if word in line:
                    results.append(f"TXT - {file_name}, Line {line_no}:
{line.strip()}")
    return results

def search_pdf(file_path, word, logical_operator=None, regex=False):
    results = []
```

```
    file_name = os.path.basename(file_path)
    reader = PdfReader(file_path)
    for page_no, page in enumerate(reader.pages, start=1):
        raw_text = page.extract_text()
        if not raw_text:
            continue
        lines = raw_text.splitlines()
        for line_no, line in enumerate(lines, start=1):
            if regex:
                if re.search(word, line):
                    results.append(f"PDF - {file_name}, Page {page_no},
Line {line_no}: {line.strip()}")
            elif logical_operator == "and":
                if all(w in line for w in word.split()):
                    results.append(f"PDF - {file_name}, Page {page_no},
Line {line_no}: {line.strip()}")
            elif logical_operator == "or":
                if any(w in line for w in word.split()):
                    results.append(f"PDF - {file_name}, Page {page_no},
Line {line_no}: {line.strip()}")
            else:
                if word in line:
                    results.append(f"PDF - {file_name}, Page {page_no},
Line {line_no}: {line.strip()}")
    return results

def search_excel(file_path, word, logical_operator=None, regex=False):
    results = []
    file_name = os.path.basename(file_path)
    xls = pd.ExcelFile(file_path)
    for sheet_name in xls.sheet_names:
        sheet = xls.parse(sheet_name)
        for row_idx, row in sheet.iterrows():
            for col_idx, cell in enumerate(row):
                cell_value = str(cell)
                if regex:
                    if re.search(word, cell_value):
                        results.append(f"Excel - {file_name}, Sheet
{sheet_name}, Row {row_idx+1}, Column {col_idx+1}: {cell_value}")
                elif logical_operator == "and":
                    if all(w in cell_value for w in word.split()):
                        results.append(f"Excel - {file_name}, Sheet
{sheet_name}, Row {row_idx+1}, Column {col_idx+1}: {cell_value}")
```

```
                elif logical_operator == "or":
                    if any(w in cell_value for w in word.split()):
                        results.append(f"Excel - {file_name}, Sheet
{sheet_name}, Row {row_idx+1}, Column {col_idx+1}: {cell_value}")
                else:
                    if word in cell_value:
                        results.append(f"Excel - {file_name}, Sheet
{sheet_name}, Row {row_idx+1}, Column {col_idx+1}: {cell_value}")
    return results

def search_html(file_path, word, logical_operator=None, regex=False):
    results = []
    file_name = os.path.basename(file_path)
    with open(file_path, "r", encoding="utf-8") as file:
        soup = BeautifulSoup(file, "html.parser")
        text = soup.get_text()
        lines = text.splitlines()
        for line_no, line in enumerate(lines, start=1):
            if regex:
                if re.search(word, line):
                    results.append(f"HTML - {file_name}, Line {line_no}:
{line.strip()}")
            elif logical_operator == "and":
                if all(w in line for w in word.split()):
                    results.append(f"HTML - {file_name}, Line {line_no}:
{line.strip()}")
            elif logical_operator == "or":
                if any(w in line for w in word.split()):
                    results.append(f"HTML - {file_name}, Line {line_no}:
{line.strip()}")
            else:
                if word in line:
                    results.append(f"HTML - {file_name}, Line {line_no}:
{line.strip()}")
    return results

def search_in_all_files(word, logical_operator=None, regex=False,
file_type="all"):
    results = []
    for root, _, files in os.walk(DIRECTORY):
        for file_name in files:
            file_path = os.path.join(root, file_name)
```

```
            # Filtrer par type de fichier
            if file_type != "all" and not
file_name.endswith(f".{file_type}"):
                continue

            if file_name.endswith(".txt"):
                results.extend(search_txt(file_path, word,
logical_operator, regex))
            elif file_name.endswith(".pdf"):
                results.extend(search_pdf(file_path, word,
logical_operator, regex))
            elif file_name.endswith(".xlsx"):
                results.extend(search_excel(file_path, word,
logical_operator, regex))
            elif file_name.endswith(".html"):
                results.extend(search_html(file_path, word,
logical_operator, regex))
    return results

def handle_client(client_socket, client_address):
    print(f"[Server] New connection from {client_address}")
    try:
        client_socket.send("Welcome to the search server! Type 'q' to
quit.\n".encode('utf-8'))

        while True:
            client_msg = client_socket.recv(BUFFER_SIZE).decode('utf-8')
            if not client_msg:
                break

            if client_msg.lower() == ENDING_MSG:
                break

            if client_msg.startswith("search:"):
                _, word, search_type, file_type = client_msg.split(":")
                results = search_in_all_files(word,
logical_operator=search_type, regex=(search_type == "regex"),
file_type=file_type)

                response = f"Found {len(results)} matches:\n" +
"\n".join(results)
                client_socket.send((response +
END_MARKER).encode('utf-8'))
```

```
            else:
                client_socket.send("Invalid command.".encode('utf-8'))

    except Exception as e:
        client_socket.send(f"Error: {e}".encode('utf-8'))
    finally:
        client_socket.close()

def start_server():
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind((SERVER_HOST, SERVER_PORT))
    server_socket.listen(5)
    print(f"[Server] Listening on {SERVER_HOST}:{SERVER_PORT}")

    while True:
        client_socket, client_address = server_socket.accept()
        client_handler = threading.Thread(target=handle_client,
args=(client_socket, client_address))
        client_handler.start()

if __name__ == "__main__":
    start_server()
```

Maintenant sans les boutons regex/or/and

```python
import socket
import threading
import os
import re
import pandas as pd
from PyPDF2 import PdfReader
from bs4 import BeautifulSoup

# Constants
SERVER_HOST = "127.0.0.1"
SERVER_PORT = 52300
BUFFER_SIZE = 1024
ENDING_MSG = "q"
END_MARKER = "<<END>>"
DIRECTORY = "./data"  # Directory containing files

def search_txt(file_path, word, logical_operator=None, regex=False):
    results = []
    file_name = os.path.basename(file_path)
    with open(file_path, "r", encoding="utf-8") as file:
        for line_no, line in enumerate(file, start=1):
            if regex:
                if re.search(word, line):
                    results.append(f"TXT - {file_name}, Line {line_no}:
{line.strip()}")
            elif logical_operator == "and":
                if all(w in line for w in word.split()):
                    results.append(f"TXT - {file_name}, Line {line_no}:
{line.strip()}")
            elif logical_operator == "or":
                if any(w in line for w in word.split()):
                    results.append(f"TXT - {file_name}, Line {line_no}:
{line.strip()}")
            else:
                if word in line:
                    results.append(f"TXT - {file_name}, Line {line_no}:
{line.strip()}")
    return results

def search_pdf(file_path, word, logical_operator=None, regex=False):
    results = []
    file_name = os.path.basename(file_path)
```

```
    reader = PdfReader(file_path)
    for page_no, page in enumerate(reader.pages, start=1):
        raw_text = page.extract_text()
        if not raw_text:
            continue
        lines = raw_text.splitlines()
        for line_no, line in enumerate(lines, start=1):
            if regex:
                if re.search(word, line):
                    results.append(f"PDF - {file_name}, Page {page_no},
Line {line_no}: {line.strip()}")
            elif logical_operator == "and":
                if all(w in line for w in word.split()):
                    results.append(f"PDF - {file_name}, Page {page_no},
Line {line_no}: {line.strip()}")
            elif logical_operator == "or":
                if any(w in line for w in word.split()):
                    results.append(f"PDF - {file_name}, Page {page_no},
Line {line_no}: {line.strip()}")
            else:
                if word in line:
                    results.append(f"PDF - {file_name}, Page {page_no},
Line {line_no}: {line.strip()}")
    return results

def search_excel(file_path, word, logical_operator=None, regex=False):
    results = []
    file_name = os.path.basename(file_path)
    xls = pd.ExcelFile(file_path)
    for sheet_name in xls.sheet_names:
        sheet = xls.parse(sheet_name)
        for row_idx, row in sheet.iterrows():
            for col_idx, cell in enumerate(row):
                cell_value = str(cell)
                if regex:
                    if re.search(word, cell_value):
                        results.append(f"Excel - {file_name}, Sheet
{sheet_name}, Row {row_idx+1}, Column {col_idx+1}: {cell_value}")
                elif logical_operator == "and":
                    if all(w in cell_value for w in word.split()):
                        results.append(f"Excel - {file_name}, Sheet
{sheet_name}, Row {row_idx+1}, Column {col_idx+1}: {cell_value}")
                elif logical_operator == "or":
```

```
                    if any(w in cell_value for w in word.split()):
                        results.append(f"Excel - {file_name}, Sheet
{sheet_name}, Row {row_idx+1}, Column {col_idx+1}: {cell_value}")
                else:
                    if word in cell_value:
                        results.append(f"Excel - {file_name}, Sheet
{sheet_name}, Row {row_idx+1}, Column {col_idx+1}: {cell_value}")
    return results

def search_html(file_path, word, logical_operator=None, regex=False):
    results = []
    file_name = os.path.basename(file_path)
    with open(file_path, "r", encoding="utf-8") as file:
        soup = BeautifulSoup(file, "html.parser")
        text = soup.get_text()
        lines = text.splitlines()
        for line_no, line in enumerate(lines, start=1):
            if regex:
                if re.search(word, line):
                    results.append(f"HTML - {file_name}, Line {line_no}:
{line.strip()}")
            elif logical_operator == "and":
                if all(w in line for w in word.split()):
                    results.append(f"HTML - {file_name}, Line {line_no}:
{line.strip()}")
            elif logical_operator == "or":
                if any(w in line for w in word.split()):
                    results.append(f"HTML - {file_name}, Line {line_no}:
{line.strip()}")
            else:
                if word in line:
                    results.append(f"HTML - {file_name}, Line {line_no}:
{line.strip()}")
    return results

def search_in_all_files(word, logical_operator=None, regex=False,
file_type="all"):
    if logical_operator is None:
        if regex:
            logical_operator = "regex"
        elif " " in word:
            logical_operator = "and"
        else:
```

```
            logical_operator = "keyword"

    results = []
    for root, _, files in os.walk(DIRECTORY):
        for file_name in files:
            file_path = os.path.join(root, file_name)
            if file_type != "all" and not
file_name.endswith(f".{file_type}"):
                continue

            if file_name.endswith(".txt"):
                results.extend(search_txt(file_path, word,
logical_operator, regex))
            elif file_name.endswith(".pdf"):
                results.extend(search_pdf(file_path, word,
logical_operator, regex))
            elif file_name.endswith(".xlsx"):
                results.extend(search_excel(file_path, word,
logical_operator, regex))
            elif file_name.endswith(".html"):
                results.extend(search_html(file_path, word,
logical_operator, regex))
    return results

def handle_client(client_socket, client_address):
    print(f"[Server] New connection from {client_address}")
    try:
        client_socket.send("Welcome to the search server! Type 'q' to
quit.\n".encode('utf-8'))

        while True:
            client_msg = client_socket.recv(BUFFER_SIZE).decode('utf-8')
            if not client_msg:
                break

            if client_msg.lower() == ENDING_MSG:
                break

            if client_msg.startswith("search:"):
                _, word, search_type, file_type = client_msg.split(":")
                results = search_in_all_files(word,
logical_operator=search_type, regex=(search_type == "regex"),
file_type=file_type)
```

```
                response = f"Found {len(results)} matches:\n" +
"\n".join(results)
                client_socket.send((response +
END_MARKER).encode('utf-8'))
            else:
                client_socket.send("Invalid command.".encode('utf-8'))

    except Exception as e:
        client_socket.send(f"Error: {e}".encode('utf-8'))
    finally:
        client_socket.close()

def start_server():
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind((SERVER_HOST, SERVER_PORT))
    server_socket.listen(5)
    print(f"[Server] Listening on {SERVER_HOST}:{SERVER_PORT}")

    while True:
        client_socket, client_address = server_socket.accept()
        client_handler = threading.Thread(target=handle_client,
args=(client_socket, client_address))
        client_handler.start()

if __name__ == "__main__":
    start_server()
```

```
Client.py

import socket
import tkinter as tk
import threading
from tkinter import ttk

# Constants
SERVER_HOST = "127.0.0.1"
SERVER_PORT = 52300
BUFFER_SIZE = 1024
END_MARKER = "<<END>>"

def search():
   word = word_entry.get()
   search_type = search_type_var.get()
   file_type = file_type_var.get()
   request = f"search:{word}:{search_type}:{file_type}"
   threading.Thread(target=handle_search_request,
args=(request,)).start()

def handle_search_request(request):
   try:
       client_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
       client_socket.connect((SERVER_HOST, SERVER_PORT))
       client_socket.send(request.encode("utf-8"))

       response = ""
       while True:
           chunk = client_socket.recv(BUFFER_SIZE).decode("utf-8")
           response += chunk
           if END_MARKER in chunk:
               break

       response = response.replace(END_MARKER, "").strip()
       root.after(0, display_results, response)
       client_socket.close()

   except Exception as e:
       root.after(0, display_results, f"Error: {e}")

def display_results(results):
```

```
    result_text.delete(1.0, tk.END)
    result_text.insert(tk.END, results)

# Tkinter setup
root = tk.Tk()
root.title("File Search Client")

# Search Word Entry
word_label = tk.Label(root, text="Search Word:")
word_label.grid(row=0, column=0)
word_entry = tk.Entry(root, width=50)
word_entry.grid(row=0, column=1)

# Search Type
search_type_label = tk.Label(root, text="Search Type:")
search_type_label.grid(row=1, column=0)
search_type_var = tk.StringVar(value="keyword")
tk.Radiobutton(root, text="Keyword", variable=search_type_var,
value="keyword").grid(row=1, column=1)
tk.Radiobutton(root, text="AND", variable=search_type_var,
value="and").grid(row=1, column=2)
tk.Radiobutton(root, text="OR", variable=search_type_var,
value="or").grid(row=1, column=3)
tk.Radiobutton(root, text="Regex", variable=search_type_var,
value="regex").grid(row=1, column=4)

# File Type Selection
file_type_label = tk.Label(root, text="File Type:")
file_type_label.grid(row=2, column=0)
file_type_var = tk.StringVar(value="all")  # Default to search all
types
file_type_menu = ttk.Combobox(root, textvariable=file_type_var,
state="readonly")
file_type_menu['values'] = ["all", "txt", "pdf", "xlsx", "html"]
file_type_menu.grid(row=2, column=1)

# Search Button
search_button = tk.Button(root, text="Search", command=search)
search_button.grid(row=3, column=0, columnspan=5)

# Textbox for results
result_text = tk.Text(root, width=80, height=20)
result_text.grid(row=4, column=0, columnspan=5)
```

```
root.mainloop()
```

Maintenant sans les boutons regex/or/and

```python
import socket
import threading
import tkinter as tk
from tkinter import ttk
import re

# Constants
SERVER_HOST = "127.0.0.1"
SERVER_PORT = 52300
BUFFER_SIZE = 1024
END_MARKER = "<<END>>"

# Handle the server response
def handle_server_response(client_socket):
    response = ""
    while True:
        chunk = client_socket.recv(BUFFER_SIZE).decode("utf-8")
        if END_MARKER in chunk:
            response += chunk.replace(END_MARKER, "")
            break
        response += chunk
    return response

# Handle the search request
def handle_search_request(request):
    try:
        client_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
        client_socket.connect((SERVER_HOST, SERVER_PORT))
        client_socket.send(request.encode("utf-8"))

        response = handle_server_response(client_socket)
        result_text.delete(1.0, tk.END)
        result_text.insert(tk.END, response)
        client_socket.close()
    except Exception as e:
        result_text.delete(1.0, tk.END)
        result_text.insert(tk.END, f"Error: {e}")

# Search logic
def search():
```

```
    word = word_entry.get().strip()
    file_type = file_type_var.get()

    # Automatic detection of search mode
    if re.search(r"[+*?|()\[\]{}]", word):  # If regex-like characters
are detected
        search_type = "regex"
    elif " " in word:
        search_type = "and"  # Default to AND for multiple words
    else:
        search_type = "keyword"

    request = f"search:{word}:{search_type}:{file_type}"
    threading.Thread(target=handle_search_request,
args=(request,)).start()

# GUI setup
root = tk.Tk()
root.title("File Search Client")

# Word entry
word_label = tk.Label(root, text="Search Word:")
word_label.grid(row=0, column=0, padx=5, pady=5)
word_entry = tk.Entry(root, width=50)
word_entry.grid(row=0, column=1, padx=5, pady=5)

# File type dropdown
file_type_label = tk.Label(root, text="File Type:")
file_type_label.grid(row=1, column=0, padx=5, pady=5)
file_type_var = tk.StringVar(value="all")
file_type_menu = ttk.Combobox(root, textvariable=file_type_var,
state="readonly")
file_type_menu["values"] = ["all", "txt", "pdf", "xlsx", "html"]
file_type_menu.grid(row=1, column=1, padx=5, pady=5)

# Search button
search_button = tk.Button(root, text="Search", command=search)
search_button.grid(row=2, column=0, columnspan=2, pady=10)

# Results text box
result_text = tk.Text(root, width=80, height=20)
result_text.grid(row=3, column=0, columnspan=2, padx=5, pady=5)
```

```
root.mainloop()
```